5-Stage Pipelined RISC-V CPU with Cache

Design, Implementation, and Verification

Verilog HDL • Icarus Verilog • GTKWave • Python

Sungkyun (Ian) Cho

McMaster University • Electrical and Computer Engineering

1. Introduction

1.1 Motivation

Modern processor design is fundamentally shaped by two competing forces: the desire to execute instructions as fast as possible, and the physical reality that memory is far slower than the processor that accesses it. This gap — often called the memory wall or the von Neumann bottleneck — is not merely a theoretical concern. It is observable in simulation: when a processor stalls waiting for data, every downstream stage in the pipeline sits idle, burning cycles doing nothing.

The motivation for this project was to make these concepts concrete through implementation. Reading about pipeline hazards, cache miss penalties, and data forwarding in a textbook conveys the mechanism but not the intuition. Building them in RTL — watching the stall signal assert in GTKWave, seeing the PC freeze for five cycles on an instruction cache miss, verifying that a forwarded value arrives at the ALU input one cycle ahead of when the register file would have provided the stale value — makes the architecture real in a way that no diagram fully achieves.

There is also a broader motivation rooted in modern computing trends. The von Neumann bottleneck that becomes obvious when building a pipelined CPU is the same bottleneck that motivates in-memory computing architectures for neural network inference. When a deep learning model loads weights from memory one by one to perform a matrix multiply, the majority of energy and time is spent on data movement rather than computation. Understanding this at the RTL level provides the hardware intuition that underlies current research in near-memory computing, processing-in-memory, and efficient accelerator design.

1.2 Project Overview

This document describes the complete design and implementation of a 5-stage pipelined RISC-V CPU in Verilog HDL, including:

- A single-cycle RV32I reference CPU as the starting point
- A 5-stage pipelined CPU with IF, ID, EX, MEM, and WB stages
- A data forwarding unit to resolve Read-After-Write (RAW) hazards without stalling
- A hazard detection unit to handle load-use hazards requiring pipeline stalls
- Direct-mapped instruction and data caches with a simulated main memory
- A Python validation script for automated register file verification

The toolchain used throughout was Icarus Verilog for simulation, GTKWave for waveform analysis, and Python 3 for post-simulation validation.

1.3 RISC-V ISA Background

RISC-V is an open-standard instruction set architecture (ISA) based on reduced instruction set computer (RISC) principles. The base integer instruction set, RV32I, defines 32-bit instructions operating on 32 general-purpose registers (x0 through x31), where x0 is hardwired to zero.

Instructions fall into six encoding formats:

R-type	Register-register operations (ADD, SUB, AND, OR, SLT)
I-type	Immediate operations and loads (ADDI, LW, JALR)
S-type	Stores (SW)
B-type	Conditional branches (BEQ, BNE)
U-type	Upper immediate (LUI, AUIPC)
J-type	Unconditional jumps (JAL)

The clean and regular encoding of RISC-V makes it well-suited to a pipelined implementation: every instruction is the same size, register fields always appear in the same place, and immediate values are always sign-extended from the same bit positions. This regularity simplifies the decode stage and the hazard detection logic.

2. Theoretical Background

2.1 Pipelining

A pipelined processor overlaps the execution of multiple instructions by dividing the data path into discrete stages, each handling one phase of instruction execution. In a 5-stage pipeline, five instructions can be in flight simultaneously — one in each stage — ideally achieving a throughput of one instruction per clock cycle (IPC = 1) in the steady state.

The five stages are:

IF (Instruction Fetch)	Read the next instruction from instruction memory using the current PC.
ID (Instruction Decode)	Decode the instruction, read source registers, generate control signals, and compute the sign-extended immediate.
EX (Execute)	Perform the ALU operation or compute the memory address. Resolve branch/jump target.
MEM (Memory Access)	Read or write data memory for load and store instructions.
WB (Write Back)	Write the result back to the destination register in the register file.

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) latch all signals between stages so that each stage can operate on a different instruction each clock cycle. Without these registers, a change in one stage would propagate combinationally through all downstream stages, destroying the pipelining effect.

2.2 Data Hazards and Forwarding

A data hazard occurs when an instruction depends on the result of a preceding instruction that has not yet completed. In a 5-stage pipeline, a result computed in the EX stage is not written back to the register file until the WB stage, two cycles later. If the very next instruction tries to read that register in the ID stage, it will read a stale value.

The solution is data forwarding (also called bypassing): rather than waiting for the value to be written to the register file, the forwarding unit detects the dependency and routes the result directly from a later pipeline stage back to the EX-stage input mux. Two forwarding paths are needed:

- EX/MEM → EX: forwards the ALU result from the EX/MEM pipeline register to the current EX stage (resolves a 1-cycle-behind dependency)
- MEM/WB → EX: forwards the result from the MEM/WB pipeline register to the current EX stage (resolves a 2-cycle-behind dependency)

When both conditions are true simultaneously (two pending writes to the same register), the EX/MEM forwarding path takes priority, as it carries the more recent value.

2.3 Load-Use Hazards

Forwarding resolves most RAW hazards, but there is one case it cannot handle: the load-use hazard. When a load instruction (lw) is immediately followed by an instruction that reads the loaded register, the data is not available until after the MEM stage of the load — but the dependent instruction needs it at the start of its EX stage, which overlaps with the MEM stage of the load.

The only solution is to stall the pipeline for one cycle. The hazard detection unit:

1. Freezes the PC (it does not increment, so the same instruction is re-fetched)
2. Freezes the IF/ID pipeline register (so the fetched instruction is not lost)
3. Injects a NOP (bubble) into the ID/EX register, preventing the dependent instruction from proceeding for one cycle

After one stall cycle, the loaded value is available from the MEM/WB forwarding path and execution proceeds normally.

2.4 Control Hazards

When a branch or jump instruction is executed, the pipeline has already begun fetching the sequential next instruction(s) — which are incorrect if the branch is taken. This project uses a flush-on-taken strategy: when a branch resolves in the EX-stage, the PC is redirected to the branch target and any instructions that were incorrectly fetched are squashed by zeroing their pipeline registers (inserting bubbles).

2.5 The Memory Hierarchy and Cache Design

The gap between processor speed and main memory speed is the defining challenge of modern computer architecture. A processor can execute instructions at one per clock cycle, but a main memory access may require tens to hundreds of cycles. Caches exploit spatial and temporal locality to bridge this gap: by keeping recently used data close to the processor in fast SRAM, most memory accesses can be served in a single cycle.

A direct-mapped cache assigns each memory address to exactly one cache line based on the index bits of the address. The tag bits are stored alongside the data to distinguish between different addresses that map to the same index. A valid bit indicates whether the cache line holds meaningful data.

On a cache hit, data is returned in the same cycle with no pipeline stall. On a cache miss, the pipeline stalls (all upstream registers are frozen) while the cache fetches the missing line from main memory. The number of stall cycles per miss equals the main memory latency, which in this implementation is 3 cycles.

Separate instruction and data caches (a split-cache or Harvard-style arrangement at the L1 level) avoid a structural hazard that would otherwise arise: the IF stage and the MEM stage would both need to access memory on the same cycle if a unified cache were used.

3. Implementation

3.1 Step 1: Single-Cycle CPU

The project began with a single-cycle RV32I CPU — a clean reference implementation where every instruction completes in one clock cycle. This provided a verified datapath to reason from before introducing pipeline complexity.

The single-cycle CPU comprises the following modules:

- imem / dmem: instruction and data memory as Verilog register arrays, initialized via $readmemh from a hex file
- regfile: 32-entry register file with synchronous write and asynchronous read; x0 hardwired to zero
- alu: performs ADD, SUB, AND, OR, SLT, and pass-through operations based on a 4-bit control code
- control: generates all datapath control signals (reg_write, alu_src, mem_write, mem_read, mem_to_reg, branch, jal, alu_op) from the opcode and funct fields
- imm_decode: sign-extends all five RISC-V immediate formats (I, S, B, U, J) based on the opcode

The GTKWave waveform below shows the single-cycle CPU executing the test program. The PC advances every cycle, instruction values appear in the instr bus, and ALU results are visible in alu_result.

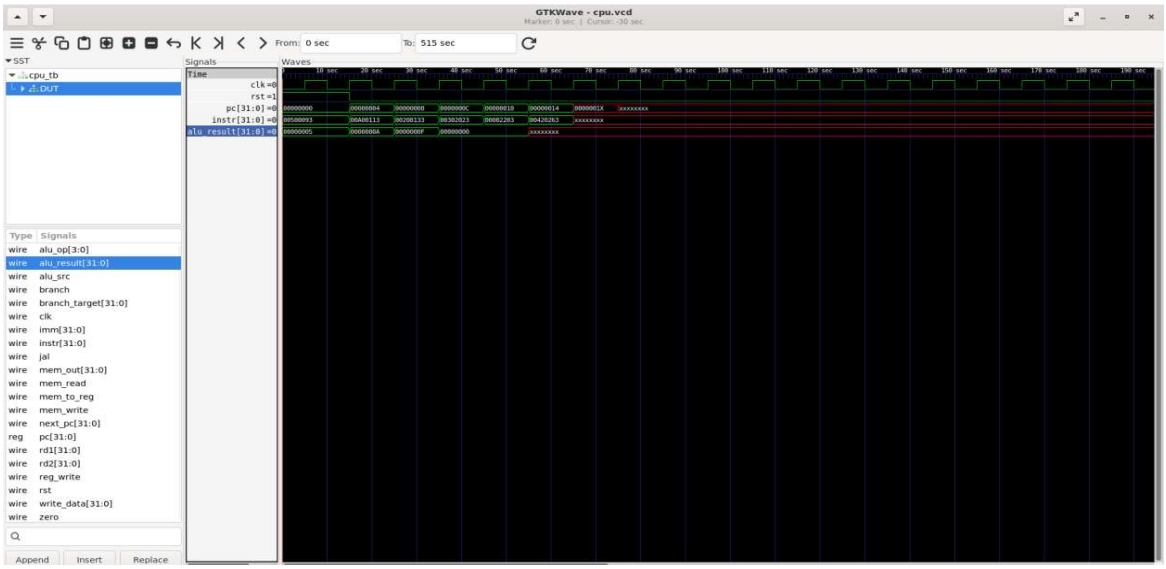

Figure 1: GTKWave waveform of the single-cycle CPU. Signals shown include clk, rst, pc[31:0], instr[31:0], and alu_result[31:0]. The PC increments each cycle and instructions are decoded immediately.

3.2 Step 2: 5-Stage Pipelined CPU

The single-cycle datapath was split into five pipeline stages by inserting four sets of pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB. Each register bank latches all signals that need to travel forward to the next stage on every rising clock edge.

At this point the CPU was structurally complete but functionally broken — data hazards caused incorrect results on any instruction that read a register written by an immediately preceding instruction. This was the expected starting state before adding hazard handling.

The pipelined waveform below shows the pipeline registers advancing in lockstep. The ifid_instr register shows instructions entering the pipeline, idex_rs1 shows source register indices in the decode/execute boundary, and exmem_alu_result shows results passing through to the memory stage.

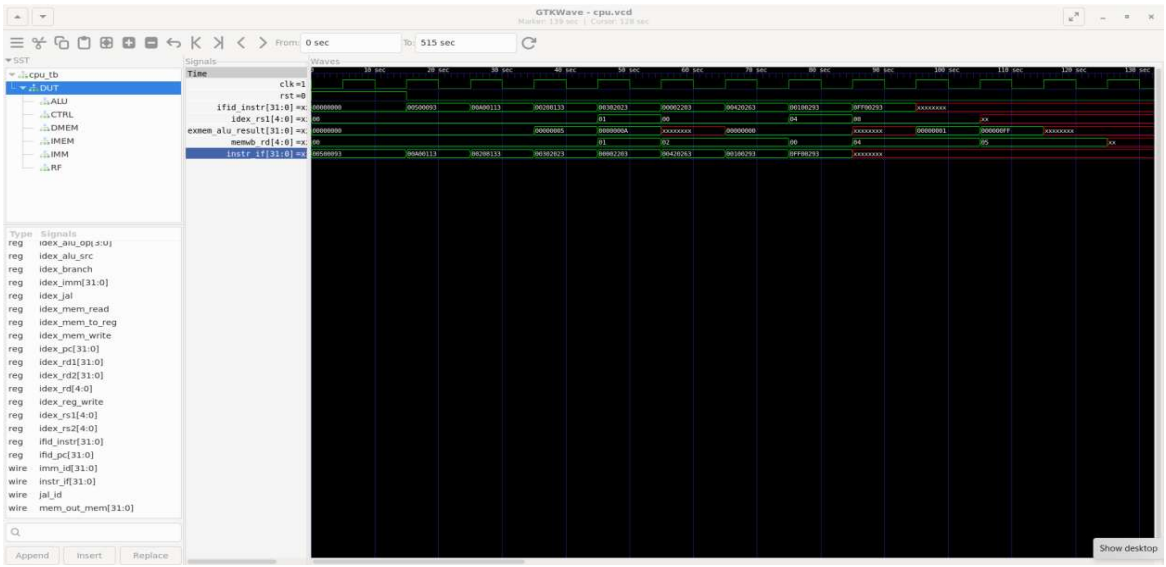

Figure 2: GTKWave waveform of the pipelined CPU. Pipeline registers IF/ID, ID/EX, EX/MEM, and MEM/WB are visible. ifid_instr, idex_rs1[4:0], exmem_alu_result[31:0], memwb_rd[4:0], and instr_if[31:0] show instructions flowing through all five stages.

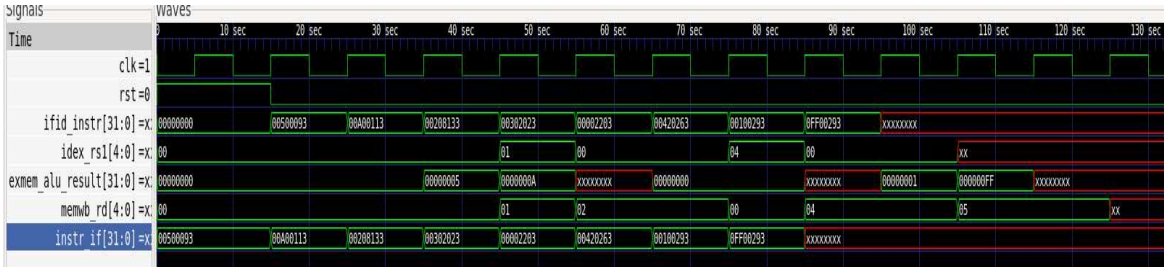

Figure 3: Zoomed pipeline register view showing key forwarding signals. The exmem_alu_result transitions (00000005, 0000000A, xxxxxxxx) correspond to the addi x1 and addi x2 results propagating through the EX/MEM stage. The memwb_rd register tracks destination register indices advancing through the pipeline.

3.3 Step 3: Hazard Handling

3.3.1 Data Forwarding Unit

The forwarding unit (forward.v) compares the destination register fields of the EX/MEM and MEM/WB pipeline registers against the source register fields of the current ID/EX stage. When a

match is detected and the writing instruction has register-write enabled, a 2-bit forwarding select signal is set for each ALU operand:

```
ForwardA = 10: forward from EX/MEM (ALU result)
ForwardA = 01: forward from MEM/WB (write-back data)
ForwardA = 00: use register file output (no hazard)
```

EX/MEM forwarding takes priority over MEM/WB forwarding when both stages would write to the same destination register. This priority ensures the most recent value is used.

3.3.2 Hazard Detection Unit

The hazard detection unit (hazard.v) monitors the ID/EX stage for load instructions (mem_read asserted) and checks whether the destination register matches source register of the instruction currently in the IF/ID stage. When a load-use hazard is detected:

- The stall signal is asserted
- The PC is frozen (write-enable disabled)
- The IF/ID register is frozen (the fetched instruction is held)
- The ID/EX register is zeroed (a NOP bubble is injected)

This correctly inserts exactly one stall cycle, after which the loaded value is available via the MEM/WB forwarding path.

3.4 Step 4: Instruction and Data Caches

3.4.1 Cache Architecture

Both caches are direct-mapped. Each cache line stores a valid bit, a tag, and one 32-bit data word. The cache lookup uses address bits to extract a tag (upper bits), an index (middle bits used to select the cache line), and a byte offset (lower 2 bits, always zero for word-aligned accesses).

A hit occurs when the valid bit is set and the stored tag matches the address tag. On a hit, data is returned combinationally in the same cycle. On a miss, a memory request is issued to the main memory module (main_mem.v), which has a 3-cycle latency modelled by a delay counter.

3.4.2 Cache Stall Logic

The unified cache_stall signal was one of the key bug fixes during development (see Section 4). In the final implementation, cache_stall is defined as:

```
assign cache_stall = (!icache_hit) || (exmem_mem_read && !dcache_hit);
```

When cache_stall is asserted, all pipeline registers are frozen — the PC does not advance, and no new instructions enter the pipeline. This correctly handles both instructions fetch misses (icache) and data memory misses (dcache) through a single unified stall path.

3.4.3 Write Policy

The data cache uses a write-through, no-write-allocate policy. Store instructions always write directly to both the data cache and the backing data memory in the same cycle. The dcache_hit signal is asserted for writes unconditionally, so store instructions never stall the pipeline.

The cache waveforms below show the complete cache behavior across 500 simulation cycles. The dcache_hit signal pulses during the store and load instructions, the req and ready signals show the main memory handshake on icache misses, and the delay_count counts the 3-cycle fill latency.

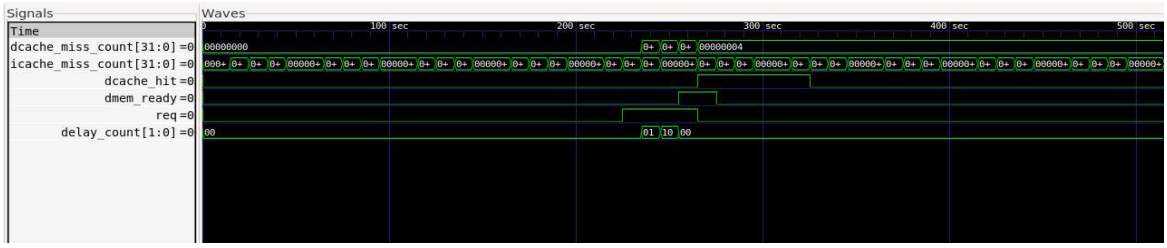

Figure 4: Cache verification waveform. dcache_miss_count shows a single increment to 4 (from the lw instruction stall cycles), icache_miss_count shows the inflated count (later corrected), dcache_hit pulses on the store and load, and delay_count shows the 3-cycle memory fill sequence (01, 10, 00).

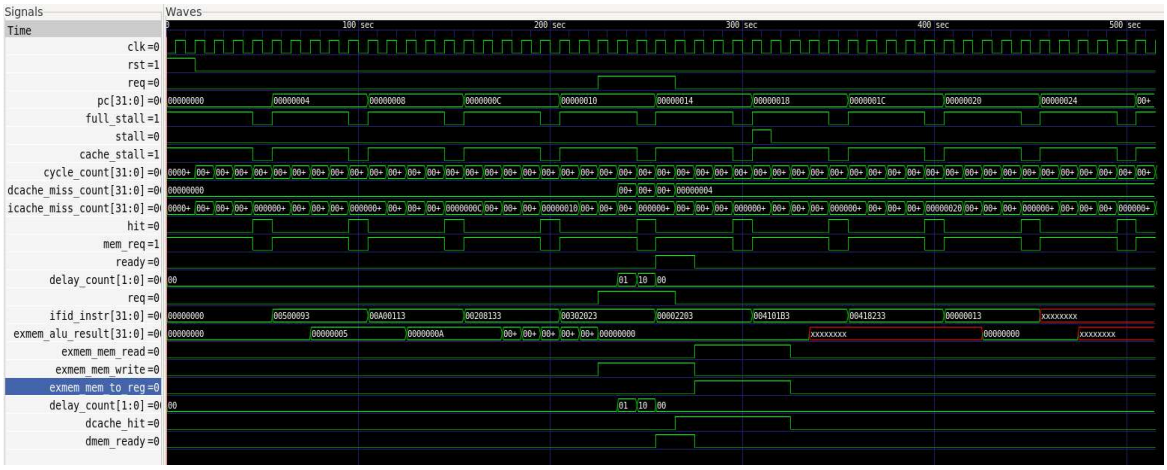

Figure 5: Full cache and pipeline signal overlay across 500 simulation cycles. Key signals include clk, rst, req, pc[31:0], full_stall, stall, cache_stall, cycle_count[31:0], dcache_miss_count[31:0], icache_miss_count[31:0], hit, mem_req, ready, delay_count[1:0], ifid_instr[31:0], exmem_alu_result[31:0], exmem_mem_read, exmem_mem_write, and dcache_hit. The cache_stall signal is clearly visible holding the pipeline during each instruction cache miss.

4. Challenges and How They Were Resolved

4.1 Bug: PC Not Frozen on Data Cache Miss

The first and most significant bug was that the PC was not being frozen during a data cache miss. The original cache_stall signal was defined as:

```
assign cache_stall = (exmem_mem_read && !dcache_hit);  // original, incorrect
```

This signal correctly froze the pipeline registers (ID/EX, EX/MEM, MEM/WB) during a dcache miss. However, the PC update block used a separate check:

```
end else if (icache_hit) begin  // original PC guard
    pc <= pc + 4;
```

The result was that during a dcache miss, the pipeline registers were frozen but the PC kept advancing. New instructions were being fetched into the front of the pipeline while the load instruction was stalled in the MEM stage — corrupting the pipeline state and producing incorrect results.

The fix was to unify both stall sources into a single signal and use it consistently everywhere:

```
assign cache_stall = (!icache_hit) || (exmem_mem_read && !dcache_hit);  //
fixed
end else if (!cache_stall) begin  // PC now frozen on both miss types
    pc <= pc + 4;
```

The three pipeline register freeze checks were also simplified from !icache_hit || cache_stall to just cache_stall, since icache miss is now subsumed in the unified signal.

4.2 Bug: Icache Miss Counter Inflated

The original miss counter incremented every cycle that icache_hit was low:

```
if (!icache_hit)
    icache_miss_count <= icache_miss_count + 1;
```

Since an icache miss spans multiple cycles (the 3-cycle main memory fill), the counter incremented 3-5 times per actual miss. For a 9-instruction test program, this produced an icache_miss_count of 133 instead of the expected ~10.

```
chos45@bob:~/riscv_cpu$ python3 validate.py
[*] Running simulation: vvp cpu_sim
[*] Simulation produced 505 output lines
[*] Found 297 writeback events

[*] Performance: 500 cycles | icache misses: 133 | dcache misses: 0

=======================================================
  Register File Validation
=======================================================
  Reg        Expected          Got
  x1     0x0000000005 0x0000000005  pass ✓
  x2     0x000000000f 0x000000000f  pass ✓
  x3     0x000000000f 0x000000000f  pass ✓
  x4     0x000000000f 0x000000000f  pass ✓
=======================================================

  ✓ ALL CHECKS PASSED
```

Figure 6: Python validation output showing icache misses: 133 — the inflated count from the original counter that incremented every stall cycle rather than once per miss.

The fix was to track the rising edge of icache_mem_req (the request signal to main memory), which pulses high exactly once at the start of each miss:

```
reg icache_req_prev;
icache_req_prev <= icache_mem_req;
if (icache_mem_req && !icache_req_prev && !exmem_jal && !exmem_jal_prev
    && !(exmem_branch && exmem_zero) && cycle_count > 4)
    icache_miss_count <= icache_miss_count + 1;
```

Several additional guard conditions were required, each discovered through waveform debugging:

- cycle_count > 4: suppresses counts during the reset window when the PC has not yet begun fetching
- !exmem_jal and !exmem_jal_prev: suppresses the spurious mem_req pulse from the wrong-path fetch one cycle before and one cycle after the jal resolves
- !(exmem_branch && exmem_zero): suppresses similar artifacts from taken branches

A debug testbench (cpu_tb_debug.v) was written to print a log line on every cycle where the miss counter incremented. This confirmed exactly which cycles were being counted:

```
MISS_COUNT cy=5 pc=00000004 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=10 pc=00000008 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=15 pc=0000000c imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=20 pc=00000010 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=25 pc=00000014 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=30 pc=00000018 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=36 pc=0000001c imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=41 pc=00000020 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=46 pc=00000024 imem_req=1 exmem_jal=0 exmem_jal_prev=0
MISS_COUNT cy=51 pc=00000028 imem_req=1 exmem_jal=0 exmem_jal_prev=0
```

Figure 7: MISS_COUNT debug output. Each line corresponds to one increment of the icache_miss_count register. The 10 entries correspond to 10 unique instruction addresses fetched for the first time: 9 real program instructions plus one speculative wrong-path fetch at address 0x24 caused by the jal branch penalty.

```
[*] Running simulation: vvp cpu_sim
[*] Simulation produced 505 output lines
[*] Found 297 writeback events

[*] Performance: 500 cycles | icache misses: 10 | dcache misses: 0

=======================================================
  Register File Validation
=======================================================
  Reg        Expected          Got
  x1     0x0000000005 0x0000000005  pass ✓
  x2     0x000000000f 0x000000000f  pass ✓
  x3     0x000000000f 0x000000000f  pass ✓
  x4     0x000000000f 0x000000000f  pass ✓
=======================================================

  ✓ ALL CHECKS PASSED
```

Figure 8: Python validation output showing icache misses: 10 — correct icache miss count from the original counter that incremented once per miss.

The final count of 10 is correct: 9 cold misses for the 9 program instructions, plus 1 speculative fetch of an uncached address during the branch penalty of the jal instruction. This is genuine cache behavior, not a counting error.

4.3 Debugging Methodology

The primary debugging tool throughout was GTKWave. For pipeline issues, the key technique was to add pipeline-register contents to the waveform and trace an instruction's progress cycle by cycle through each stage. Stall signals and forwarding select signals were added to make hazard handling directly observable.

For the cache miss counter debugging, a custom $display statement was added to the testbench to print a log line every cycle that the miss counter incremented. This converted an abstract counter value into a per-cycle trace that could be directly matched against the waveform.

For high-level correctness checking, the Python validation script was used. It parses writeback events from the simulation log, reconstructs the register file state, and compares against expected values — providing a fast, repeatable pass/fail check that removed the need to manually inspect waveforms for functional correctness.

5. Results and Verification

5.1 Test Program

The test program exercises the key hazard and cache scenarios in sequence:

addi x1, x0, 5	Immediate load: x1 = 5
addi x2, x0, 10	Immediate load: x2 = 10
add x2, x1, x2	RAW hazard: x2 depends on x1 (forwarded from EX/MEM)
sw x3, 0(x0)	Store: writes x3=0 to memory address 0
lw x4, 0(x0)	Load: triggers load-use stall + dcache miss
add x3, x2, x4	Load-use: x4 depends on lw result (1 stall cycle)
add x4, x3, x4	RAW: x3 depends on previous add (forwarded)
nop	No operation
jal x0, 0	Halt: infinite loop to itself

Expected final register state: x1 = 5, x2 = 15, x3 = 15, x4 = 15. All other registers remain 0.

5.2 Register File Validation

The Python validation script parsed 297 writeback events from the 505-line simulation log and reconstructed the final register file. All four non-zero registers matched their expected values:

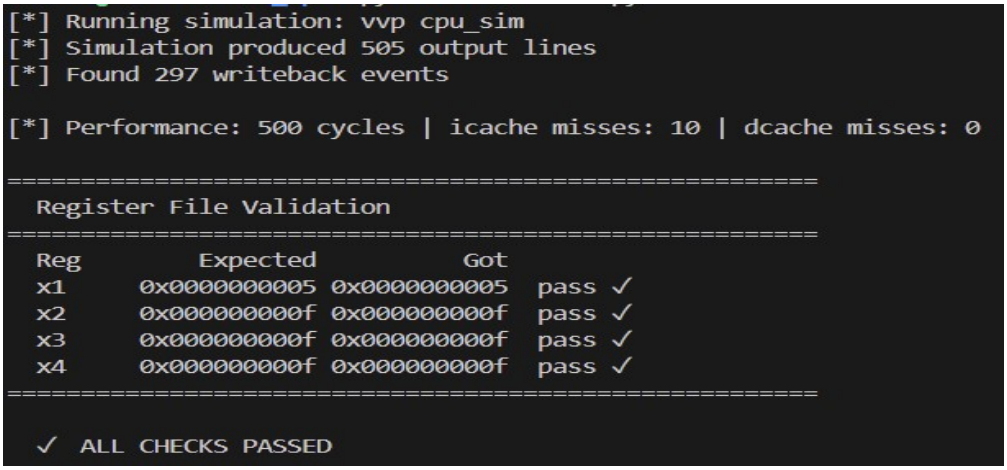

Figure 7: Python validation output after all bug fixes. icache misses: 10 (corrected from 133). All four register checks pass: x1=5, x2=15, x3=15, x4=15.

5.3 Cache Performance

The simulation ran for 500 cycles, during which the icache warmed up on the first pass through the program (10 cold misses) and then served all subsequent fetches as hits. The second pass through the loop ran at 1 cycle per instruction — confirmed by observing the PC advancing every cycle rather than every 5 cycles.

Total simulation cycles	500
icache cold misses (first pass)	10
dcache misses	0 (write-through, store hits)
Second-pass fetch latency	1 cycle/instruction (all icache hits)
Load-use stall cycles	1 (for the lw instruction)
Branch flush cycles	2 (for the jal on first pass)

6. Conclusions

6.1 What Was Built

This project produced a fully functional 5-stage pipelined RISC-V CPU in Verilog, verified through simulation and automated register checking. The final implementation handles data forwarding for back-to-back RAW hazards, load-use stalls, branch flushes, instruction cache misses, and data cache misses — all through a unified stall control path.

6.2 What Was Learned

The most important insight from this project was how concretely observable the memory hierarchy is in simulation. An instruction cache miss produces a multi-cycle stall that is clearly visible in GTKWave: the PC freezes, the pipeline registers contents hold, and no new results appear at the writeback stage. Five cycles of the processor doing nothing to fetch a single instruction makes the von Neumann bottleneck tangible in a way that diagrams do not.

A second key lesson was the importance of unified control signals. The cache stall bug (Section 4.1) arose precisely because two separate signals — one for the PC and one for the pipeline registers — were supposed to fire together but did not. Collapsing both into a single cache_stall signal and using it everywhere removed an entire class of potential inconsistency.

A third lesson was the value of targeted debug instrumentation. Adding a single $display line that printed when the miss counter incremented turned an opaque counter value into a readable per-cycle log. The root cause of the inflated count (speculative wrong-path fetches from branch resolution) was immediately apparent from the log output, whereas waveform inspection alone would have required much more manual analysis.

6.3 Broader Implications

Building this CPU from scratch makes several trends in computer architecture intuitive rather than abstract. The pipeline stalls from cache misses — visible as the PC holding the same value for five cycles — directly illustrate why memory bandwidth is the bottleneck in data-intensive workloads. A neural network inference kernel loading weights from memory is performing the same operation, just at a larger scale: many sequential cache misses, each stalling the datapath while data moves from memory to the processor.

In-memory computing architectures address this by performing computation inside or adjacent to the memory array, eliminating the data movement that causes the stalls. Having built the hardware that exhibits the bottleneck, the motivation for near-memory and processing-in-memory designs is no longer abstract.

6.4 Future Work

Several improvements could extend this project:

- Branch prediction: a static predict-not-taken or a 2-bit saturating counter predictor would reduce the branch penalty from 2 flush cycles to 0 on correctly predicted branches
- Set-associative caches: replacing direct-mapped caches with 2-way set-associative caches would reduce conflict miss rates at the cost of a tag comparison mux and a replacement policy (LRU or pseudo-LRU)
- Cache size and associativity sweep: measuring IPC across a range of cache parameters against a larger benchmark program would produce the kind of performance analysis used to characterize real cache hierarchies
- Exception handling: adding support for illegal instruction and misaligned access exceptions would make the CPU more complete as a RISC-V implementation